Ten Hidden Bits and a Rank-One Fix: A Response to Kryptonite-3.0

Group Member One [*1] **Group Member Two** [*1] **Group Member Three** [*1] **Group Member Four** [*1]

Abstract

Quinn & Luther (2026) claim that the Kryptonite-3.0 datasets defeat polynomial basis expansion, more training data and a purpose-built foundation model, KryptoNet-40. We show that every Kryptonite task is the same Boolean function of only ten of the n input bits, a two-bit parity combined with two four-bit gadgets, observed with 3.9% label noise, so the best achievable accuracy is about 96.1%. This function has almost no low-degree Fourier weight, which explains the mock paper's 55% results, yet it is a degree-6 polynomial threshold function: the mock paper's own model on ten coordinates selected from training data reaches 95.8% on Kryptonite-40, and an MLP trained from scratch 95.3%. Learning curves show that more data helps every model that can represent the function, while the quadratic model studied by the mock paper is provably capped at 56.5%. Kryptonite-40-Adapt is KryptoNet-40's pretraining task with one input coordinate moved, so LoRA of rank one on the first layer (552 trainable parameters) adapts KryptoNet-40 to 95.4% at two-thirds of the per-step operations of full fine-tuning.

1. Introduction

Challenge datasets audit claims about machine learning by exposing what current methods cannot do (Chollet, 2019), but only if each failure is attributed correctly: to approximation (no function in the hypothesis class fits), estimation (too few samples to find it) or optimisation (training does not find it). The Kryptonite-3.0 mock paper (Quinn & Luther, 2026) attributes three failures to machine learning itself: polynomial basis expansion up to degree 10 stays below 59%, said to contradict universal approximation; quadrupling the data barely helps, so the difficulty is "intrinsic"; and the foundation model KryptoNet-40 is "worse than a coin flip" on Kryptonite-40-Adapt, even when probed.

We treat each claim as a falsifiable hypothesis. We first measure the structure of the data with Fourier analysis on the Boolean hypercube (O'Donnell, 2014), which reveals which coordinates matter and how hard the labels are for polynomials, kernels and networks, and then run controlled experiments that change one factor at a time.

Section 2 develops the theory, including low-rank adaptation (LoRA; Hu et al., 2022) and its operation count; Section 3 states four hypotheses and our safeguards; Section 4 reports the anatomy of the data, polynomial models with and without feature selection, nine learners on five shared splits, learning curves, and the adaptation of KryptoNet-40.

All three claims fail. Every task is one fixed 10-bit Boolean function hidden among $n - 10$ distractor bits. Polynomials represent it from degree 6, so universal approximation is not contradicted; the mock paper's models lack samples for their $\binom{n+d}{d}$ features and succeed once irrelevant coordinates are removed. More data helps every model that can represent the function, and KryptoNet-40 needs only a rank-one change to its first layer, which LoRA finds. We expect about 96% on all four hidden sets and release all code and predictions.[1]

2. Methodology

2.1. Data Model and Fourier Analysis

We observe $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ with $\mathbf{x} \in \mathbb{R}^n$, $y \in \{0,1\}$, and write $s = 2y - 1$ and $\mathbf{b} = \operatorname{sgn}(\mathbf{x}) \in \{\pm 1\}^n$. Every $f : \{\pm 1\}^n \to \{\pm 1\}$ has a unique multilinear (Fourier) expansion

$$f(\mathbf{b}) = \sum_{S \subseteq [n]} \hat{f}(S)\, \chi_S(\mathbf{b}), \qquad \chi_S(\mathbf{b}) = \prod_{i \in S} b_i, \tag{1}$$

with $\hat{f}(S) = \mathbb{E}_{\mathbf{b}}[f(\mathbf{b})\chi_S(\mathbf{b})]$ under the uniform distribution and $\sum_S \hat{f}(S)^2 = 1$. We call $W^{=d}[f] = \sum_{|S|=d} \hat{f}(S)^2$ the weight at degree d, and $\mathrm{Inf}_i[f] =$

[1]Department of Computing, Imperial College London, London, United Kingdom. Correspondence to: Group Member One <first.last@imperial.ac.uk>.

Coursework. For review by instructors of Mathematics for Machine Learning (70015) . Do not distribute.

[1]GitLab repository: https://gitlab.doc.ic.ac.uk/GROUP/kryptonite-response, commit COMMITHASH.

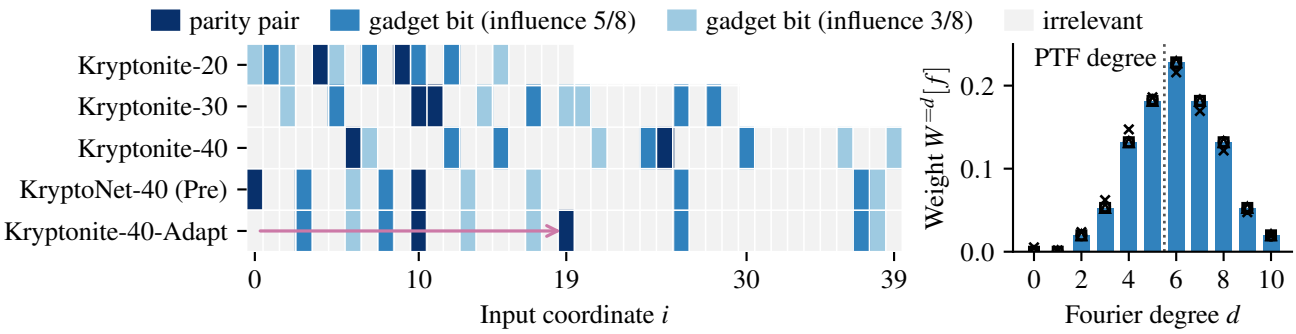


Figure 1. Anatomy of Kryptonite-3.0. *Left:* coordinates selected from training data, coloured by their influence on the estimated label function. Every task reads exactly 10 coordinates with the same roles; Adapt is KryptoNet-40's pretraining task with the parity bit at coordinate 0 moved to 19 (arrow). *Right:* Fourier weight of the label function by degree (bars: Kryptonite-40; markers: the other four tasks, which coincide); the function is a polynomial threshold function (PTF) of degree 6.

$\sum_{S\ni i}\hat{f}(S)^2$, the probability that flipping b_i flips f, the influence of coordinate i. With labels flipped at rate η, sample means of $s\chi_S(\mathbf{b})$ estimate $(1-2\eta)\hat{f}(S)$ with standard error $N^{-1/2}$. By (1), a function of k coordinates is *exactly* a polynomial of degree $\le k$ in them.

2.2. Polynomial Logistic Regression

Logistic regression with basis ϕ models $P(y=1\mid\mathbf{x})=\sigma(\mathbf{w}^\top\phi(\mathbf{x}))$ and minimises $C\sum_i \ell(y^{(i)},\mathbf{w}^\top\phi(\mathbf{x}^{(i)}))+\frac{1}{2}\|\mathbf{w}\|^2$ with the log-loss ℓ; $C=0.85$ in the mock paper. On ±1 bits every monomial of degree $\le d$ reduces to a parity χ_S with $|S|\le d$, so the model is a polynomial threshold function (PTF) $\operatorname{sgn}(\sum_{|S|\le d} w_S\chi_S)$. The mock paper fits it in the dual with the normalised kernel $k(\mathbf{b},\mathbf{b}')=\sum_{|S|\le d} c_{|S|}\chi_S(\mathbf{b})\chi_S(\mathbf{b}')/Z$ and Newton's method (Rasmussen & Williams, 2006); on k selected bits we fit the identical model in the primal, with at most 2^k features $\sqrt{c_{|S|}/Z}\,\chi_S$.

Proposition 1 (the quadratic model is capped)**.** *Let $f=\chi_P\, g$ with $P=\{p,q\}$ and g independent of b_p, b_q, and let labels be flipped with rate η. Over all polynomials of degree ≤ 2 in the bits (or in the raw features), the population logistic loss is minimised by $c^\star x_p x_q$, whose accuracy is $\frac{1}{2}\big(1+|\hat{f}(P)|\big)(1-2\eta)+\eta$.*

The proof (Appendix A) uses a symmetry: negating x_q negates the label, so the unique minimiser can only use monomials of odd degree in both x_p and x_q. Logistic regression is consistent for this minimiser, so no amount of data takes the quadratic model beyond it: 56.5% for Kryptonite.

2.3. Kernels, Trees and Neural Networks

Kernels. The approximation theorems cited by the mock paper (De Branges, 1959; Hornik, 1991) are existence statements and say nothing about sample size. For dot-product kernels on the hypercube the parities are eigenfunctions, and with $N\asymp n^{\ell+\delta}$ samples kernel ridge regression learns the projection of the target onto degree-$\le\ell$ polynomials and nothing more (Ghorbani et al., 2021; Mei et al., 2022). Here $N\approx 10^4$ and $n\ge 20$ give $\log N/\log n<3$, so even a degree-10 kernel effectively fits only the degree-≤ 2 part of f. RBF kernels and k-NN suffer likewise, as irrelevant coordinates dominate their distances; on the 10 relevant bits, by contrast, the whole function space has dimension $2^{10}\ll N$.

Trees. Random forests (Breiman, 2001) and gradient boosting split greedily on one coordinate at a time, so when every bit is independent of the label ($W^{=1}[f]=0$) the first splits are uninformative.

Neural networks. An MLP $f_\theta(\mathbf{x})=\mathbf{w}_L^\top\mathrm{ReLU}(\mathbf{W}_{L-1}\cdots\mathrm{ReLU}(\mathbf{W}_1\mathbf{x}+\mathbf{b}_1))$ is trained on mini-batches $\mathcal{B}_t$ with AdamW (Kingma & Ba, 2015; Loshchilov & Hutter, 2019): with $\mathbf{g}_t=\nabla_\theta\frac{1}{|\mathcal{B}_t|}\sum_{i\in\mathcal{B}_t}\ell(y^{(i)},f_\theta(\mathbf{x}^{(i)}))$ and bias-corrected moment estimates $\hat{\mathbf{m}}_t,\hat{\mathbf{v}}_t$ of $\mathbf{g}_t$ and $\mathbf{g}_t^2$,

$$\theta_{t+1}=\theta_t-\alpha\big(\hat{\mathbf{m}}_t/(\sqrt{\hat{\mathbf{v}}_t}+\epsilon)+\lambda\theta_t\big),\qquad(2)$$

with learning rate α, weight decay λ, batch size $|\mathcal{B}_t|$ and the number of epochs (set by early stopping) as hyperparameters. Unlike a kernel, the network can align its first layer with the relevant coordinates. For two-layer networks trained by SGD, the sample complexity of a sparse function is governed by its *leap*, the largest number of new coordinates that must be found at once along its Fourier support, and scales as $\tilde{O}(n^{\max(\mathrm{Leap}-1,1)})$ (Abbe et al., 2022; 2023). Kryptonite has leap 2 (Section 4.1): polynomial in n with an exponent that does not grow with the degree, unlike the $\binom{n+6}{6}$ features of a degree-6 kernel.

2.4. Low-Rank Adaptation and Its Operation Count

KryptoNet-40 is a ReLU MLP with weights $\mathbf{W}_1\in\mathbb{R}^{512\times 40}$, $\mathbf{W}_2,\mathbf{W}_3,\mathbf{W}_4\in\mathbb{R}^{512\times 512}$ and $\mathbf{w}_5\in\mathbb{R}^{512}$ (809,473 pa-

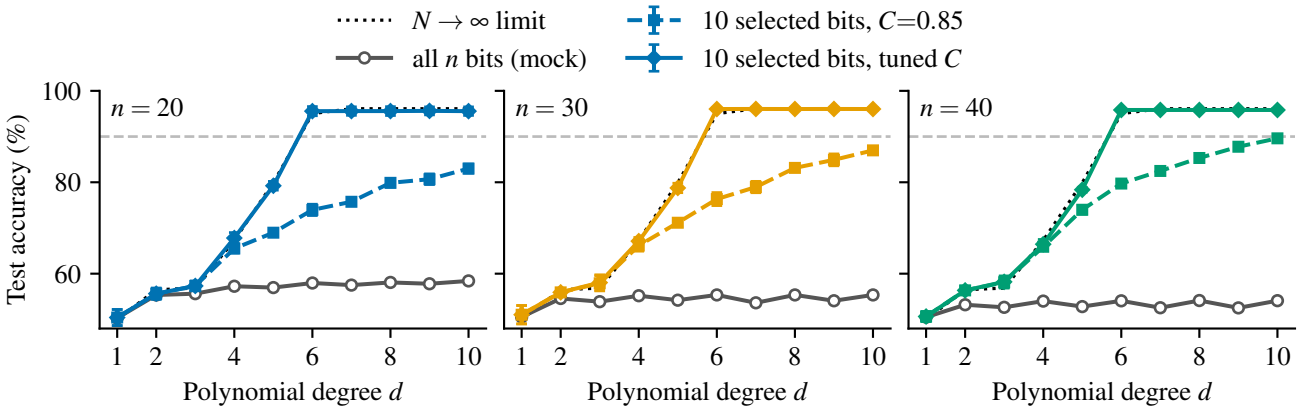


Figure 2. Polynomial logistic regression against degree d. Grey: the mock paper's model on all n bits, reproduced on its split. Coloured: the same model on the 10 coordinates selected from training data, with the mock paper's $C = 0.85$ (dashed) or with C chosen on validation data (solid); mean and 95% CI over 5 splits. Dotted: infinite-data limit of degree-d logistic regression. Dashed grey: the 90% target.

Table 1. Test accuracy (%) on Kryptonite-n: mean ± 95% CI over the same 5 splits for every model; bold: above the 90% target. "Representation" states how the model's input representation is obtained. Time: seconds per fit at $n = 40$ on two CPU cores, including hyper-parameter search.

Model	Representation	$n = 20$	$n = 30$	$n = 40$	Time (s)
Linear logistic regression	fixed	50.4 ± 0.4	50.3 ± 1.1	50.2 ± 0.6	<0.1
Quadratic LR (mock paper)	fixed	53.9 ± 1.6	52.9 ± 1.2	53.1 ± 0.6	8.4
Degree-10 kernel LR (mock paper)	fixed	58.5 ± 1.1	54.8 ± 1.1	54.0 ± 0.6	45
k-nearest neighbours	fixed	56.9 ± 1.1	53.3 ± 0.8	53.0 ± 0.4	2.1
RBF-kernel SVM	fixed	57.7 ± 1.1	55.1 ± 0.7	53.2 ± 0.6	272
Random forest	greedy splits	52.1 ± 1.3	51.8 ± 1.1	51.9 ± 0.9	20
Gradient-boosted trees	greedy splits	56.5 ± 1.8	52.3 ± 1.6	54.2 ± 7.3	0.2
MLP, all n inputs (ours)	learned	$\mathbf{93.8 \pm 0.7}$	$\mathbf{94.9 \pm 0.8}$	$\mathbf{95.3 \pm 0.2}$	139
Sparse polynomial LR (ours)	selected	$\mathbf{95.6 \pm 0.9}$	$\mathbf{96.0 \pm 0.4}$	$\mathbf{95.8 \pm 0.2}$	38

rameters). For a layer $\mathbf{h} = \mathbf{W}\mathbf{x} + \mathbf{b}$ with $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA (Hu et al., 2022) *freezes* $\mathbf{W}$ and $\mathbf{b}$ and trains only a low-rank correction:

$$\mathbf{h} = \mathbf{W}\mathbf{x} + \mathbf{b} + \tfrac{\alpha}{r}\,\mathbf{B}\mathbf{A}\mathbf{x}, \qquad \mathbf{A} \in \mathbb{R}^{r \times d_{\text{in}}},\ \mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r}. \tag{3}$$

$\mathbf{A}$ is initialised randomly (Kaiming-uniform) and $\mathbf{B} = \mathbf{0}$, so training starts exactly at the pretrained network while gradients still reach $\mathbf{B}$. The rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$ caps the number of independent directions in which the layer can change, $\operatorname{rank}(\Delta\mathbf{W}) \le r$ for $\Delta\mathbf{W} = \frac{\alpha}{r}\mathbf{B}\mathbf{A}$, and sets the trainable parameters to $r(d_{\text{in}} + d_{\text{out}})$ instead of $d_{\text{out}}d_{\text{in}} + d_{\text{out}}$; α (we use $\alpha = r$) decouples the update scale from r. After training, $\mathbf{W} + \Delta\mathbf{W}$ is merged once, so inference costs nothing extra.

Operation count. Let $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times N}$ hold a batch and $\mathbf{G} = \partial L/\partial \mathbf{H} \in \mathbb{R}^{d_{\text{out}} \times N}$ be the gradient from the layer above; an $n \times m$ by $m \times k$ product costs nmk, and elementwise operations are not counted. *Full fine-tuning* computes $\mathbf{W}\mathbf{X}$ ($d_{\text{out}}d_{\text{in}}N$) forward, and $\nabla_{\mathbf{W}} = \mathbf{G}\mathbf{X}^\top$ ($d_{\text{out}}Nd_{\text{in}}$) and the input gradient $\mathbf{W}^\top\mathbf{G}$ ($d_{\text{in}}d_{\text{out}}N$) backward. *LoRA evaluated right to left*, $\mathbf{B}(\mathbf{A}\mathbf{X})$, computes $\mathbf{W}\mathbf{X}$, $\mathbf{U} = \mathbf{A}\mathbf{X}$ (rNd_{in}) and $\mathbf{B}\mathbf{U}$ ($d_{\text{out}}rN$) forward, and $\nabla_{\mathbf{B}} = \mathbf{G}\mathbf{U}^\top$ ($d_{\text{out}}Nr$), $\nabla_{\mathbf{U}} = \mathbf{B}^\top\mathbf{G}$ ($rd_{\text{out}}N$), $\nabla_{\mathbf{A}} = \nabla_{\mathbf{U}}\mathbf{X}^\top$ (rNd_{in}) and $\mathbf{W}^\top\mathbf{G} + \mathbf{A}^\top\nabla_{\mathbf{U}}$ ($d_{\text{in}}d_{\text{out}}N + d_{\text{in}}rN$) backward; the frozen $\mathbf{W}$ needs no weight gradient. Hence

$$\underbrace{2Nd_{\text{in}}d_{\text{out}} + 3rN(d_{\text{in}} + d_{\text{out}})}_{\text{LoRA, } \mathbf{B}(\mathbf{A}\mathbf{X})} \quad \text{vs.} \quad \underbrace{3Nd_{\text{in}}d_{\text{out}}}_{\text{full}}, \tag{4}$$

a ratio of $\frac{2}{3} + \frac{r(d_{\text{in}}+d_{\text{out}})}{d_{\text{in}}d_{\text{out}}}$, below 1 exactly when $r < d_{\text{in}}d_{\text{out}}/(3(d_{\text{in}}+d_{\text{out}}))$ (85 for a 512×512 layer). The first layer needs no input gradient, giving $Nd_{\text{in}}d_{\text{out}} + rN(2d_{\text{in}} + 3d_{\text{out}})$ against $2Nd_{\text{in}}d_{\text{out}}$, a ratio near $\frac{1}{2}$. *The order of evaluation matters*: left to right, $(\mathbf{B}\mathbf{A})\mathbf{X}$ forms $\mathbf{M} = \mathbf{B}\mathbf{A}$ ($d_{\text{out}}rd_{\text{in}}$) and the full-size gradient $\nabla_{\mathbf{M}} = \mathbf{G}\mathbf{X}^\top$ ($d_{\text{out}}Nd_{\text{in}}$) before $\nabla_{\mathbf{B}} = \nabla_{\mathbf{M}}\mathbf{A}^\top$ and $\nabla_{\mathbf{A}} = \mathbf{B}^\top\nabla_{\mathbf{M}}$ ($d_{\text{out}}d_{\text{in}}r$ each), costing $3Nd_{\text{in}}d_{\text{out}} + 3rd_{\text{in}}d_{\text{out}}$, *more* than full fine-tuning. PyTorch's operation counter confirms all counts exactly (Table 2). Because the forward pass and the input gradient remain, LoRA saves at most a third of the

Table 2. Operations ($\times 10^6$) of one training step with batch $N = 64$, measured with PyTorch's `FlopCounterMode` (equal to Eq. 4 and its variants), and trainable parameters.

Method	Forward	Backward	Trainable
Hidden layer $\mathbf{W}_2$ *(*512×512*),* $r = 4$			
Full fine-tuning	16.78	33.55	262,656
LoRA, $\mathbf{B}(\mathbf{AX})$	17.04	17.30	4,096
LoRA, $(\mathbf{BA})\mathbf{X}$	17.83	35.65	4,096
First layer $\mathbf{W}_1$ *(*512×40*),* $r = 1$			
Full fine-tuning	1.31	1.31	20,992
LoRA, $\mathbf{B}(\mathbf{AX})$	1.35	0.07	552
LoRA, $(\mathbf{BA})\mathbf{X}$	1.33	1.35	552
Whole KryptoNet-40			
Full fine-tuning	51.68	102.04	809,473
LoRA on $\mathbf{W}_1$, $r = 1$	51.71	50.43	552

operations of a hidden layer; its large savings are in trainable parameters, optimiser state (two Adam moments per trainable parameter) and per-task storage.

Proposition 2 (rank one suffices; probes cannot)**.** *Suppose* $y_{\text{adapt}}(\mathbf{x}) = y_{\text{pre}}(\Pi\mathbf{x})$*, where* Π *swaps coordinates* 0 *and* 19*. (a) With* $\mathbf{w}_j$ *the* j*-th column of* $\mathbf{W}_1$*, the rank-one update* $\mathbf{W}_1' = \mathbf{W}_1 + (\mathbf{w}_{19} - \mathbf{w}_0)(\mathbf{e}_0 - \mathbf{e}_{19})^\top = \mathbf{W}_1\Pi$ *makes KryptoNet-40 exactly as accurate on Adapt as on its pre-training task. (b) If a frozen representation* $\mathbf{h}(\mathbf{x})$ *does not depend on* b_{19} *and* $s = b_{19}z$ *with* z *independent of* b_{19}*, every classifier of* $\mathbf{h}$ *(zero-shot head, linear or MLP probe) has accuracy exactly* $\frac{1}{2}$*.*

Part (a) swaps two columns of $\mathbf{W}_1$; part (b) holds because b_{19} stays uniform given $(\mathbf{h}, z)$ (Appendix A).

3. Experimental Design

We test four hypotheses against the mock paper's claims. **H1** (approximation vs. estimation): the mock paper's polynomial model fails through sample complexity, not expressiveness, so the same model on coordinates selected from training data reaches $\geq 90\%$ from degree 6. **H2** (feature learning): learners that adapt their representation (an MLP, feature selection) reach $\geq 90\%$ for every n; kernels, k-NN and greedy trees stay near the degree-2 ceiling. **H3** (data): capable models improve steeply with N and cross 90% within the released data, while quadratic logistic regression plateaus at its population limit. **H4** (adaptation): LoRA adapts KryptoNet-40 to $\geq 90\%$ with few trainable parameters, while zero-shot use and frozen-representation probes are at chance.

Validity. (1) *Shared repeated splits*: every model on Kryptonite-n uses the same five stratified 60/20/20 train/validation/test splits (seeds 0–4), so comparisons are paired; Adapt uses five 64/16/20 splits with 800 test points, as in the mock paper. We report the mean and a 95% Student-t interval over splits. (2) *No test leakage*: feature selection, hyper-parameters, regularisation and early stopping use only training and validation data, and each test set is used once per model and split. (3) *Reproduction and controls*: we reproduce the mock paper's numbers with its code and split (e.g. 58.4% for $n = 20$ at degree 10), change one factor at a time, and include zero-shot use, probes, training from scratch and full fine-tuning as controls for KryptoNet-40. (4) *Sampling error*: a 95% binomial interval on 800 test points is ± 3.5 points.

Feature selection. Greedy forward selection starts from the coordinate pair whose bits give a lookup table with the lowest validation log-loss (single bits carry no signal), adds one coordinate at a time, and stops when the validation log-loss stops decreasing. A lookup table on the selected bits is the plug-in Bayes classifier and equals a full-degree polynomial in them.

Models. Our MLP has width 512, with depth $\in \{2, 3\}$ and dropout $\in \{0, 0.2\}$ chosen on validation data; it is trained by Eq. (2) with $\alpha = 10^{-3}$, $\lambda = 0$, batches of 128 and early stopping on validation accuracy (patience 30, at most 300 epochs). KryptoNet-40 methods use batches of 64 and choose $\alpha \in \{3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}\}$ on validation data. Baselines are listed in Appendix C.

4. Experimental Results

4.1. Anatomy of Kryptonite-n

Every feature is a noisy bit, $x_i = b_i + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, 0.1^2)$ and $|x_i| > 0.49$, so $\mathbf{b} = \operatorname{sgn}(\mathbf{x})$ is exact; no single bit correlates with the label. Greedy selection finds exactly 10 coordinates for every n, the same 10 on all five splits (Figure 1). On them, Kryptonite-20/30/40 and the function computed by KryptoNet-40 (queried with synthetic inputs) share one label function up to relabelling coordinates: their majority-vote truth tables agree on 99.7–100% of the 2^{10} patterns. Its Fourier coefficients take only the values $\pm\frac{9}{64}, \pm\frac{3}{64}, \pm\frac{1}{64}$ and 0, and it factorises as

$$f(\mathbf{b}) = b_p\, b_q\, h_A(\mathbf{b}_A)\, h_B(\mathbf{b}_B), \qquad |A| = |B| = 4, \quad (5)$$

a parity of two bits XOR-ed with two 4-bit "gadgets" (Appendix B). Labels disagree with f at rate $\eta = 3.9\%$, so the Bayes accuracy is 96.1%. Equation (5) explains the difficulty: $W^{\leq 1}[f] = 0$, $W^{\leq 2}[f] = (\frac{9}{64})^2 \approx 0.02$ from the single pair $P = \{p, q\}$, and 85% of the weight lies at degrees 4–8. A linear program shows that f is a PTF of degree 6 but not 5. Its Fourier support is a staircase of leap 2: first P, then $P \cup \{l\}$ for every gadget bit l, all with $|\hat{f}| \geq \frac{3}{64}$. Finally, KryptoNet-40 scores 96.0% on Adapt once coordinate 19 is copied into slot 0 (49.5% without),

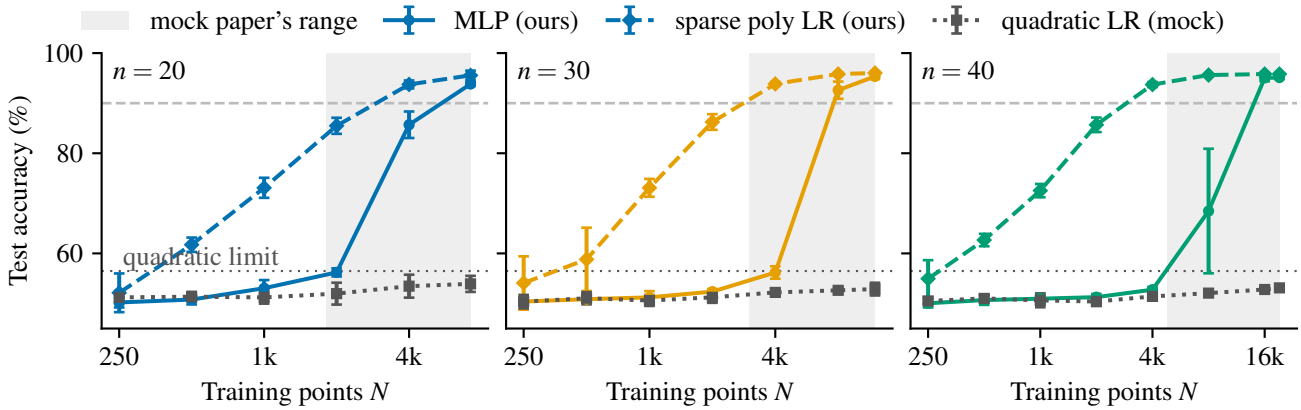


Figure 3. Learning curves: test accuracy against the number of training points (log scale), mean and 95% CI over 5 splits with fixed validation and test sets; line style marks the model and colour the dataset. Grey band: the range studied by the mock paper (25–100% of the training split). Dotted: the infinite-data limit of quadratic logistic regression (Proposition 1). Dashed: the 90% target.

confirming the premise of Proposition 2.

4.2. Polynomial Models: Estimation, Not Approximation

Our reproduction of the mock paper's model on all n bits matches its Figure 1, ending at 58.4%, 55.4% and 54.1% for $n = 20, 30, 40$ at degree 10 (Figure 2). Changing only the input to the 10 selected coordinates lifts the same model, with the same $C = 0.85$, to 83.0%, 87.0% and 89.6%. The remaining gap is the regulariser: the normalised kernel gives high-degree parities tiny feature scales, which $C = 0.85$ shrinks heavily. With C chosen on validation data, accuracy jumps between degrees 5 and 6, exactly at the PTF degree of f, and reaches 95.6%, 96.0% and 95.8% at degree 10. The infinite-data limit agrees: degree 6 suffices (95.0%, and 96.1% from degree 7; regularised fits can slightly exceed it, as log-loss is not accuracy), degree 2 is capped at 56.5% (Proposition 1), and the all-bit models stay near that cap at every nominal degree, as kernel theory predicts for $\log N / \log n < 3$. **H1 is supported**: Kryptonite does not contradict universal approximation; its difficulty for kernels lies in the $n - 10$ distractor coordinates.

4.3. Which Learners Succeed

Table 1 compares nine learners on identical splits. All learners with a fixed representation score between 50% and 59%: linear logistic regression is at chance because $W^{=1}[f] = 0$; the quadratic and degree-10 polynomial models, k-NN and the RBF-kernel SVM sit near the degree-2 ceiling; and the tree ensembles reach only 52–57% because greedy splits receive no marginal signal. The two learners that adapt their representation clear the target for every n: our MLP on all n raw inputs, without feature engineering, reaches 93.8%, 94.9% and 95.3%, and sparse polynomial logistic regression reaches 95.6%, 96.0% and 95.8%, close to the Bayes accuracy. **H2 is supported**.

4.4. More Data Helps Capable Models

Figure 3 extends the mock paper's Figure 2 over two orders of magnitude and to capable models. Quadratic logistic regression approaches, but cannot exceed, its population limit of 56.5%, so its flat curve measures approximation error, not the value of data. Models that can represent f improve sharply: the sparse polynomial model crosses 90% at about 2,900, 2,800 and 2,900 training points for $n = 20, 30, 40$, and the MLP at about 5,500, 7,600 and 14,000. The MLP shows the plateau-then-jump typical of SGD on sparse parities (Barak et al., 2022), and its jump lies *inside* the range the mock paper examined, which missed it by scaling only the quadratic model. The data it needs grows with the number of distractors (2.6-fold from $n = 20$ to 40), far more slowly than the $\binom{n+6}{6}$ features of a degree-6 kernel (41-fold). **H3 is supported**: the difficulty is not intrinsic, and the target is reached within the released data.

4.5. Adapting KryptoNet-40

Zero-shot use (50.0%), the linear probe (50.9%) and the MLP probe (49.9%) are at chance (Figure 4; all methods in Table 3), as Proposition 2(b) predicts: a linear probe recovers b_0 from KryptoNet-40's last hidden layer with 97% accuracy but b_{19} with only 58%. The mock paper's "worse than random" scores of 46–50% are within the ± 3.5-point sampling interval around 50%; the representation does not destroy information, it lacks one bit. Training the same architecture from scratch also stays at chance (52.0%), so pretraining matters. Full fine-tuning reaches 90.2%, and LoRA does better with far fewer parameters: rank one on $\mathbf{W}_1$ alone reaches 95.4%, matching the analytic swap of Proposition 2(a) (95.6%), and every LoRA configuration we tried reaches at least 94.8%. LoRA also recovers the

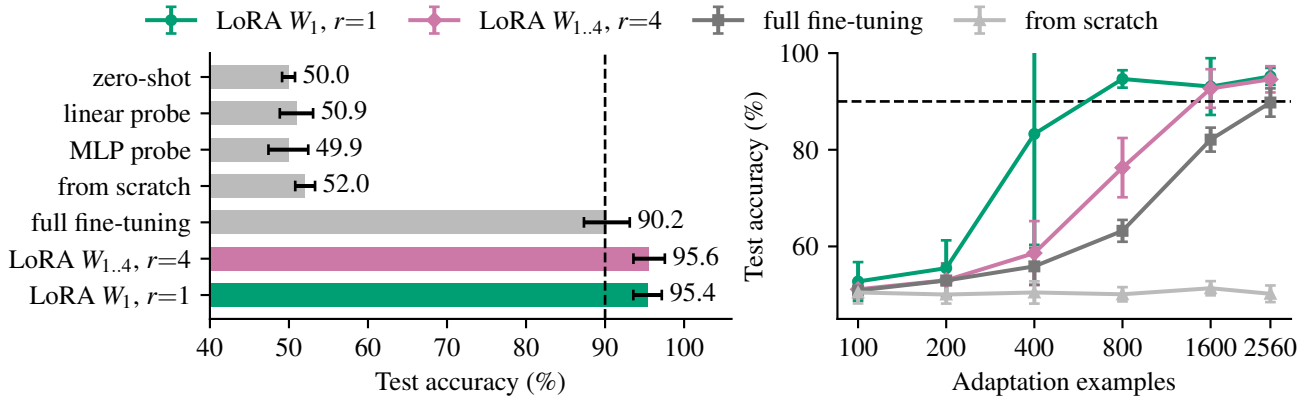


Figure 4. Adapting KryptoNet-40 to Kryptonite-40-Adapt. *Left:* test accuracy (mean and 95% CI over 5 splits of 800 test points) of the frozen-representation methods, the same architecture trained from scratch, full fine-tuning (809,473 trainable parameters), LoRA on $\mathbf{W}_1$–$\mathbf{W}_4$ with $r = 4$ (14,496) and LoRA on $\mathbf{W}_1$ with $r = 1$ (552). *Right:* test accuracy against the number of adaptation examples. Dashed: the 90% target.

mechanism: on split 0, the learned $\mathbf{A} \in \mathbb{R}^{1\times 40}$ has weights -0.81 and $+0.78$ on coordinates 0 and 19 and at most 0.04 in absolute value elsewhere, i.e. it moves the input of coordinate 0 to coordinate 19. With little data (Figure 4, right), rank-1 LoRA reaches 94.7% from 800 examples, where full fine-tuning reaches 63.2%. Per training step it performs 0.66 of the operations of full fine-tuning (Table 2) while training 552 instead of 809,473 parameters. **H4 is supported**.

Hidden-set predictions. For Kryptonite-20/30/40 we submit sparse polynomial logistic regression refitted on all labelled data (expected 95.6%, 96.0% and 95.8%; Table 1); it agrees with an independently trained MLP on 97.7%, 99.3% and 99.5% of the hidden points. For Adapt we submit the majority vote of five rank-1 LoRA models trained on different 80/20 splits, which agrees with the analytic swap on 99.8% of the hidden points.

5. Discussion

What we showed. The mock paper's three failures have three different causes, none of them a limit of machine learning: *estimation* for the polynomial models, whose hypothesis class contains the Bayes classifier from degree 6 but cannot be searched in n dimensions with 10^4 samples; *approximation* for the data experiment, which scaled the one model that is provably capped at 56.5%; and the *choice of adaptation* for KryptoNet-40, since no frozen representation can recover a coordinate the network ignores, while a rank-one update can.

Limitations and further experiments. Our description of the generator is empirical: verified on every labelled set, but the hidden labels cannot be inspected; the degree sweep of the all-bit kernel model used only the mock paper's split (degree 10: all splits, Table 1). We would have liked to test the leap-based scaling of the MLP over a wider range of n, and harder adaptations: in a preliminary run, LoRA on $\mathbf{W}_1$ with $r \leq 16$ could not adapt KryptoNet-40 to Kryptonite-40, where all ten relevant coordinates move at once and no single move lowers the loss—the leap phenomenon again.

Environmental impact. We logged the wall-clock time of every experiment and converted it to energy with the Green Algorithms model (Lannelongue et al., 2021), assuming two cores at 12 W, 4 GB of memory at 0.3725 W/GB and a power usage effectiveness of 1. All experiments took 4.3 h on two cores, about 109 Wh and 14 g CO_2e at the 2025 average carbon intensity of the GB grid, 128.8 g/kWh (Energy Dashboard, 2026), or 19 g at the 175.9 g/kWh estimate of Electricity Maps (2026) (Table 4). Model choice matters more than these totals: at $n = 40$ one fit of the mock paper's degree-10 kernel model takes 45 s and $O(N^3)$ operations per Newton step, our sparse polynomial model 38 s with over 40 points more accuracy. Adaptation reuses the energy spent on pretraining, favouring parameter-efficient methods (Strubell et al., 2019); rank-1 LoRA trains in seconds and stores 552 numbers per task. Cheap adaptation also eases repurposing released models for unintended uses, which releases should anticipate.

Societal perspective. Claims that machine learning is "all hype" and claims of imminent general intelligence share a failure mode, outcomes not attributed to a cause; challenge datasets serve public debate best with controls and stated uncertainty.

References

Abbe, E., Boix-Adserà, E., and Misiakiewicz, T. The merged-staircase property: a necessary and nearly sufficient condition for SGD learning of sparse functions on two-layer neural networks. In *Proceedings of the Confer-*

ence on Learning Theory (COLT), volume 178 of *PMLR*, pp. 4782–4887, 2022.

Abbe, E., Boix-Adserà, E., and Misiakiewicz, T. SGD learning on neural networks: leap complexity and saddle-to-saddle dynamics. In *Proceedings of the Conference on Learning Theory (COLT)*, volume 195 of *PMLR*, pp. 2552–2623, 2023.

Barak, B., Edelman, B. L., Goel, S., Kakade, S., Malach, E., and Zhang, C. Hidden progress in deep learning: SGD learns parities near the computational limit. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Breiman, L. Random forests. *Machine Learning*, 45(1): 5–32, 2001.

Chollet, F. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

De Branges, L. The Stone-Weierstrass theorem. *Proceedings of the American Mathematical Society*, 10(5):822–824, 1959.

Electricity Maps. Electricity grid review 2025: Great Britain. https://www.electricitymaps.com/grid-in-review-2025/great-britain, 2026. Accessed 27 October 2026.

Energy Dashboard. 2025 electricity review. https://www.energydashboard.co.uk/insights/2025-review, 2026. Accessed 27 October 2026.

Ghorbani, B., Mei, S., Misiakiewicz, T., and Montanari, A. Linearized two-layers neural networks in high dimension. *The Annals of Statistics*, 49(2):1029–1054, 2021.

Hornik, K. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Lannelongue, L., Grealey, J., and Inouye, M. Green algorithms: Quantifying the carbon footprint of computation. *Advanced Science*, 8(12):2100707, 2021.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Mei, S., Misiakiewicz, T., and Montanari, A. Generalization error of random feature and kernel methods: hypercontractivity and kernel matrix concentration. *Applied and Computational Harmonic Analysis*, 59:3–84, 2022.

O'Donnell, R. *Analysis of Boolean Functions*. Cambridge University Press, 2014.

Quinn, H. and Luther, L. Kryptonite-3.0: The final end to machine learning hype?, 2026. Mock paper, Mathematics for Machine Learning (70015) coursework prompt, Imperial College London.

Rasmussen, C. E. and Williams, C. K. I. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

Strubell, E., Ganesh, A., and McCallum, A. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3645–3650, 2019.

A. Proofs

Proposition 1. Write the noisy label as $s = f(\mathbf{b})\xi$ with $\xi \in \{\pm 1\}$, $P(\xi = -1) = \eta$, independent of $\mathbf{x}$. Let $\phi(\mathbf{x})$ collect the monomials of degree ≤ 2 (in the bits b_i or the raw features $x_i = b_i + \varepsilon_i$; for bits, $b_i^2 = 1$). The population loss $L(\mathbf{w}) = \mathbb{E}[\log(1 + e^{-s\,\mathbf{w}^\top \phi(\mathbf{x})})]$ is strictly convex because $\mathbb{E}[\phi\phi^\top] \succ 0$, so it has a unique minimiser $\mathbf{w}^\star$. Negating coordinate q, $(b_q, \varepsilon_q) \mapsto (-b_q, -\varepsilon_q)$, leaves the distribution of $\mathbf{x}$ unchanged (the bits are uniform and the noise symmetric) and maps s to $-s$, because $f = b_p b_q\, g$ and g does not involve b_q. A monomial is unchanged by this map if its degree in x_q is even and negated if it is odd. Hence $L(\mathbf{w}) = L(\tilde{\mathbf{w}})$, where $\tilde{w}_T = -w_T$ for monomials T of even degree in x_q and $\tilde{w}_T = w_T$ otherwise, and uniqueness forces $w^\star_T = 0$ for all monomials of even degree in x_q. The same holds for p, so $\mathbf{w}^\star$ is supported on monomials of odd degree in both x_p and x_q; with degree ≤ 2 the only one is $x_p x_q$. The resulting classifier is $\operatorname{sgn}(c^\star x_p x_q) = \operatorname{sgn}(c^\star)\, b_p b_q$, which agrees with $s = b_p b_q\, g\, \xi$ exactly when $g\xi = \operatorname{sgn}(c^\star)$. This has probability $\frac{1}{2}(1 + |\mathbb{E}g|)(1 - 2\eta) + \eta$ since $\operatorname{sgn}(c^\star) = \operatorname{sgn}(\mathbb{E}g)$, and $\mathbb{E}g = \mathbb{E}[f\chi_P] = \hat{f}(P)$. For Kryptonite, $|\hat{f}(P)| = \frac{9}{64}$ and $\eta = 3.9\%$ give 56.5%.

Proposition 2. (a) $\mathbf{W}_1\Pi$ is $\mathbf{W}_1$ with columns 0 and 19 exchanged, so the edited network computes the pretrained network on $\Pi\mathbf{x}$, and $\mathbf{W}_1\Pi - \mathbf{W}_1 = -\mathbf{W}_1(\mathbf{e}_0 - \mathbf{e}_{19})(\mathbf{e}_0 - \mathbf{e}_{19})^\top = (\mathbf{w}_{19} - \mathbf{w}_0)(\mathbf{e}_0 - \mathbf{e}_{19})^\top$ has rank one. (b) Given $(\mathbf{h}, z)$, b_{19} is still uniform, so $P(s = 1 \mid \mathbf{h}, z) = \frac{1}{2}$, s is independent of $\mathbf{h}$, and any classifier of $\mathbf{h}$ is correct with probability $\frac{1}{2}$.

B. The Kryptonite-3.0 Generator

For KryptoNet-40's pretraining task the parity pair is $P = \{0, 10\}$ and the gadget blocks are $A = \{3, 6, 17, 37\}$ and $B = \{8, 13, 26, 38\}$; Adapt replaces coordinate 0 by 19, and Kryptonite-20/30/40 use the relabellings printed in notebook 01. Each gadget is a 4-bit function equal to $+1$ on 5 of its 16 inputs (up to a global sign), with 6 Fourier coefficients of magnitude $\frac{3}{8}$ and 10 of magnitude $\frac{1}{8}$; products with the parity give the 36, 120 and 100 coefficients of f of magnitude $\frac{9}{64}$, $\frac{3}{64}$ and $\frac{1}{64}$. Within each block, two bits have influence $\frac{5}{8}$ and two have influence $\frac{3}{8}$.

C. Additional Details and Results

Reproducibility. Figure 1: `notebooks/01_dataset_anatomy`; Figure 2: `02_polynomial_models`; Table 1: `03_model_comparison`; Figure 3: `04_learning_curves`; Figure 4 and Table 3: `05_kryptonet_lora`; Table 2: `06_lora_operation_count`; hidden-set predictions: `07_hidden_predictions`; Table 4: `08_sustainability`. Shared code is in the package `krypto`; every number in this paper is generated from the notebooks' outputs by `report/make_numbers.py`.

Baselines. Linear logistic regression ($C = 0.85$); the mock paper's quadratic model (explicit features, SAG, $C = 0.85$, 100 iterations) and degree-10 kernel model; k-NN with $k \in \{1, 5, 15, 51, 151\}$ chosen on validation; an RBF-kernel SVM ($C = 10$, γ set by `scale`); a random forest with 300 trees; histogram gradient boosting with 500 iterations and early stopping. Selected MLP configurations: $n = 20$: depth 3, dropout 0.2; $n = 30$: depth 3, dropout 0.2; $n = 40$: depth 3, dropout 0.2.

Table 3. All KryptoNet-40 adaptation methods on Kryptonite-40-Adapt: trainable parameters and test accuracy (%), mean ± 95% CI over 5 splits of 800 test points.

Method	Parameters	Accuracy (%)
Zero-shot	0	50.0 ± 0.8
Linear probe (mock)	513	50.9 ± 2.1
MLP probe (mock)	263,169	49.9 ± 2.5
From scratch	809,473	52.0 ± 1.3
Full fine-tuning	809,473	**90.2 ± 2.9**
LoRA on $\mathbf{W}_1$, $r = 1$	552	**95.4 ± 1.8**
LoRA on $\mathbf{W}_1$, $r = 2$	1,104	**95.4 ± 1.5**
LoRA on $\mathbf{W}_1$, $r = 4$	2,208	**95.3 ± 1.7**
LoRA on $\mathbf{W}_1$, $r = 8$	4,416	**95.2 ± 1.6**
LoRA on $\mathbf{W}_{1..4}$, $r = 1$	3,624	**95.3 ± 1.7**
LoRA on $\mathbf{W}_{1..4}$, $r = 4$	14,496	**95.6 ± 2.0**
LoRA on $\mathbf{W}_{1..4}$, $r = 8$	28,992	**94.8 ± 1.8**
Analytic rank-1 swap	0	**95.6 ± 1.7**

Table 4. Wall-clock time on two CPU cores, energy and emissions of our experiments (assumptions in Section 5).

Notebook	Time (h)	Energy (Wh)	CO_2e (g)
01 Anatomy	0.00	0.1	0.0
02 Polynomials	0.66	16.9	2.2
03 Comparison	1.23	31.4	4.0
04 Curves	1.42	36.2	4.7
05 KryptoNet-40	0.84	21.5	2.8
06 Op. counts	0.00	0.0	0.0
07 Final models	0.11	2.8	0.4
Total	4.27	108.8	14.0